Group 5 Barrelfish Final Report

Chenyi (Allan) Wang, Shane Nelson, Sahil Gupta

June 6, 2026

Contents

Core Operating System Overview

Introduction

This section gives a high-level view of the core operating system we built across milestones M1 through M6. The system runs on ARMv8 (AArch64) hardware either emulated on QEMU or through a 4-core Toradex iMX8X board and is based off the Barrelfish OS. The detailed design and implementation of each subsystem are described in the milestone chapters that follow. This overview is a brief, high-level overview of the different components we implemented and how they interact with the existing Barrelfish structure.

Microkernel and Capabilities

Minimal Trusted Core

The Barrelfish kernel, called the CPU driver, is a thin privileged layer that runs independently on each core. Each core's CPU driver manages only the hardware directly beneath it(the MMU, the interrupt controller, and the scheduling of user-space dispatchers). Operations that are supposed to be executed by the kernel in usual monolithic kernels are now handled by user-space programs. As such the kernel in Barrelfish is a microkernel. Microkernels only handle essential core operations and push functionality out of the kernel.

When a user-space program needs to perform a privileged action, say map a page, create an endpoint, boot a second core, it issues a system call that names a specific capability it already holds. The kernel checks that the capability grants the requested permission and, if so, carries out the operation atomically. These type of capabilities are called partitioned capabilities.

Capabilities

Capabilities are essentially tokens that are associated with kernel operations/objects and represent the rights held over them. There are different types of capabilties that are retyped from physical memory. Namely, the following are commonly utilized within our implementation.

- **RAM** — a contiguous range of physical memory that has not yet been committed to any purpose. A process that holds a RAM capability can retype it into frames, page-table nodes, endpoints, or other objects.
- **Frame** — a mappable physical page. Can be installed in a page table or shared with another process by transferring the capability.
- **VNode** — a hardware page-table node (L0–L3 on ARMv8). Holds the mappings of a particular level of the address-space tree.
- **Endpoint** — a communication buffer in the kernel. The target of a Local Message-Passing (LMP) send.
- **Dispatcher** — the scheduler context of a running domain. Holding this capability allows pausing, resuming, or destroying the domain.
- **Kernel Control Block (KCB)** — per-core kernel state. Must be allocated and handed to the kernel before a new core can be started.

Capabilities live in per-process CSpaces, capability namespaces structured as trees of CNodes, and are never directly accessible to user code. To allocate capabilities and retype them into the correct capabilities, we acquire chunk of untyped memory, split the memory into the correct size, and then retype the capability into the desired type.

User-Space System Structure

The `init` Process

On each core a single privileged user-space process called `init` acts as the system manager. It is the first program to run after the CPU driver hands off execution, and it holds the initial RAM capabilities granted by the bootloader.

`init` is responsible for:

- Dividing physical memory among itself and the processes it spawns.
- Constructing the virtual address space of each new process and loading its ELF image.
- Providing every OS service that other processes request via RPC. Memory allocation, serial I/O, process lifecycle operations, and coordination with peer cores.

Because `init` holds the only RAM capabilities in the system, no process can obtain memory without going through it. This gives `init` complete control over resources without any kernel support.

Child Processes

All other programs run as children of `init`. A newly spawned process starts with almost nothing: a valid address space, a stack, its ELF image loaded, and the endpoint of `init`'s RPC server(which is a capability). Everything else the process needs (more memory, I/O access, knowledge of other processes) is obtained by sending requests to `init` over that channel.

Spawning, pausing, resuming, and killing processes are all `init` responsibilities, carried out through the RPC interface(described later).

Memory Management

Physical and virtual memory management are both implemented entirely in user space.

Physical Memory

`init` holds a pool of RAM capabilities passed up from the bootloader. When a process requests memory, `init`'s physical allocator carves a sub-region out of the pool, retypes it into a Frame capability, and transfers that capability over LMP to the requesting process. The requesting process then maps the frame into its own address space using the VNode capabilities it holds.

The kernel is involved only at the retyping step (to enforce the capability type system) and at the mapping step (to update the hardware MMU). Every other desicion is made entirely in user space through our code(size to allocate, which region to allocate, etc.).

Virtual Memory

Each process manages its own virtual address space. The paging library (`lib/aos/paging`) tracks which virtual address ranges are free and which are occupied, allocates and maps page-table VNodes on demand, and installs a page fault handler that lazily maps frames as they are first accessed.

Inter-Process Communication

In a microkernel, since kernel functionality is pushed out to user-space programs. This means other programs can no longer invoke these services through a simple function call or a syscall that touches shared kernel state. Rather it must send a message to the process that owns the service and wait for a reply. IPC is therefore is the fundamental mechanism through which the system functions and is required by other processes.

The system provides two IPC mechanisms, chosen based on whether the communicating parties are on the same core or on different cores.

Local Message Passing (LMP) — Intra-Core

LMP handles communication between processes on the same core. Messages are deposited into a kernel-managed endpoint buffer and the receiving process is scheduled when a message arrives. The entire RPC protocol between child processes and `init` is built on top of LMP.

User Message Passing (UMP) — Inter-Core

LMP endpoints cannot be shared across cores, so cross-core communication is implemented in user space over a shared physical frame. This makes UMP higher-latency than LMP, but it requires no kernel support and scales as cores are added.

The RPC Protocol

Since `init` owns all system resources, every process that needs memory, wants to spawn a child, or reads from serial I/O has to ask `init` for it. RPC gives these interactions a clean structure. Rather than ad-hoc messages, each service has a well-defined call and reply, making the interface between processes predictable and easy to extend.

All communication between a child process and `init` follows a unified request-reply protocol, regardless of whether the underlying transport is LMP or UMP. From a child process's perspective, making a service request looks the same whether `init` is running on the same core or a different one. The RPC library handles the details: it sends a tagged request, blocks until the matching reply arrives, and returns the result to the caller. On the `init` side, incoming messages are dispatched to the appropriate handler. If the request concerns a process on a remote core, `init` forwards it over UMP and relays the reply back transparently.

Multi-Core

The system runs on up to four cores, each hosting its own independent `init` instance with its own memory pool and process table. Cores communicate over UMP channels backed by shared physical frames, forming a fully-connected mesh. From the perspective of a user process, the system appears as a single coherent OS regardless of how many cores are active.

M2: Virtual Memory

Author: Chenyi Wang

The goal of this milestone is to enable user-level processes to manage their virtual address spaces.

Architecture Overview

Page table representation

We keep a software shadow tree of the hardware page tables so we can find any mapping by virtual address without re-walking the live tables. The data structure is struct `shadow_pt` in `include/aos/paging_types.h`, one node per hardware L0/L1/L2/L3 table, carrying the capref of that vnode, the mapping cap that installed it into its parent, and a 512-entry children array indexed by the same PTE slot that the hardware would use. The L0 root is at `paging_state.l0_shadow` and is built in `paging_init_state`. Levels below it are created lazily by `ensure_l3` the first time a mapping touches that subtree. We never pre-allocate a level we have not used.

This mirror is what makes unmap and shadow tree walks cheap. Without it we would either have to traverse the kernel page tables through the cap system, which is not part of the user-side API, or remember every pair in a list, which scales poorly.

Looking ahead, we could use a balanced shadow tree to make lookups easier. Another advantage of the shadow tree is that it would let us support superpages. A leaf node could attach directly at the L0/L1/L2 level of the tree, depending on how large the superpage is. This feature would be hard to support with other data structures.

Virtual address space tracking

The free and allocated VA regions live in a single sorted doubly linked list of struct `vregion`, rooted at `paging_state.vregions`. Each node carries base, size, an allocated flag, and pointers to its neighbors. On init the list contains one big free region from `start_vaddr` to `MAX_VA_ADDR`, which is 0x7f.

`paging_alloc` walks the list, picks the first free node large enough, splits off the wasted front for alignment using `vr_split_front` and splits off any excess at the back using `vr_split_back`, then marks the result allocated. `paging_unmap` reverses the process. It clears the runs, marks the vregion free, then coalesces with the previous and next neighbors if they are also free. `vr_coalesce` is the only place that drops nodes back to the slab. The list is kept sorted by base address as an invariant of the splits, which is what lets us walk it forwards and find any address in O(N).

We chose a linked list over a balanced tree because the working set of vregions is small in practice and the constant factor on a tree node was not worth the extra slab pressure. If a future workload pushes the list into the thousands we would switch to a red-black tree without changing any caller.

Map and unmap

paging_map_frame_attr_offset is the workhorse. It ensures the helper slabs have room, possibly triggers a root cnode refill, calls paging_alloc to reserve VA, then calls do_map to install the PTEs.

do_map chunks the requested region at L3 boundaries and issues one vnode_map per chunk. For each chunk it records a vregion_run holding the L3 shadow node, the starting slot, the page count, and the mapping cap. The runs form a per-vregion list rooted at vregion.runs. Recording the chunks instead of synthesizing them later is what makes paging_unmap simple. It walks vregion.runs, calls vnode_unmap plus cap_destroy per run, frees the run nodes back to the slab, and is done. The unmap path never re-walks the shadow tree.

paging_map_fixed_attr_offset shares do_map and the run tracking. It adds a containment check, finding the unique free vregion that fully contains [vaddr, vaddr+bytes), rejecting the request if the range overlaps any allocated region, and otherwise carving and installing. This is the path that V02 and V05 stress with their overlap-rejection requirements.

Lazy Heap Allocation

morecore_alloc only reserves virtual address space; it never touches a physical frame. The implementation in lib/aos/morecore.c calls paging_alloc with the requested size and returns the base address. The cost of a malloc is O(1) in the number of pages, independent of how big the allocation is. The book suggested implementing eager mapping first and switching to lazy later. We implemented lazy from the start because the work to back up an unmapped heap with a page fault handler was already required for the grading tests, and bolting eager mapping on top would have been throwaway code.

morecore_free walks the vregion list to find the region that contains the freed pointer, then calls paging_unmap on its base address. This means the full VA region returns to the free pool in one shot, which is fine because malloc and free always pair on the same vregion base address.

The lazy design has the consequence that any heap miss inside the bench harness shows up as a page fault, not as a malloc cost. This is exactly the property that MB3 in our benchmark relies on.

Page Fault Handler

pagefault_handler in lib/aos/paging.c is registered via set_main_handler for the main thread and via paging_init_onthread for each additional thread. The handler is small on purpose. It rejects any fault at an address below NULL_GUARD_ADDR = 32 MiB as a null dereference, walks the vregion list to find the allocated region that owns the faulting page, makes sure the helper slabs have room, allocates one BASE_PAGE_SIZE frame, builds the L3 shadow node via ensure_l3 if it does not exist yet, and installs exactly one PTE through vnode_map. The new run is pushed onto the vregion.runs list so paging_unmap can find it later.

Two non-obvious decisions shape the handler. First, the PTE is installed directly rather than through the standard map path: paging_map_fixed_attr_offset rejects already-allocated vregions, which would cause the fault to loop forever. Second, all three ensure_X_slab calls run before frame_alloc so the handler never recurses into itself by faulting during a slab refill.

Slab Allocators

We carry three slabs in struct paging_state — pt_slab for shadow_pt nodes, vr_slab for vregion nodes, and run_slab for vregion_run nodes. Each is sized differently because shadow_pt is roughly 4 KiB while vregion and vregion_run are small. The initial paging state is seeded from three file-static buffers, so the very first allocations never have to go through the slab refill path that itself needs paging.

Once those static buffers are warm, the slabs grow on demand through dedicated refill functions: ensure_vr_slab, ensure_run_slab, and ensure_pt_slab. Each calls slab_refill_no_pagefault,

which allocates a frame and maps it through `paging_map_frame_attr`. The three `ensure_X_slab` functions share a re-entrancy guard pattern: any one of them being mid-refill blocks the others from refilling too. This is required because `slab_refill_no_pagefault` calls back into `paging_map_frame_attr_offset`, which would otherwise drive each of the other two slabs to refill recursively before the outer one finishes.

Slot Allocator Refill

`paging_map_frame_attr_offset` and `paging_map_fixed_attr_offset` both peek at the single slot allocator state before they reserve a vregion. If the root cnode is down to twelve free slots, the map path triggers a manual `root_slot_allocator_refill` guarded by the state `rootca.refilling` so the refill path does not re-enter itself. The threshold of twelve was picked empirically. A single map request can chew through several slots, so we want headroom before the slot allocator itself starts demanding new slots from the root.

Limitations and follow-ups

The vregion free list is a plain linked list with linear time alloc and linear time fault handler lookup. Under heavy fragmentation this becomes the bottleneck; a balanced tree would fix it without changing callers.

Superpages are not implemented. `do_map` only emits `BASE_PAGE_SIZE` PTEs, so the flag is silently dropped. Hooking 2 MiB block PTE installation into `do_map` is the next planned extension and would also enable the MB5 superpage comparison in our benchmark plan.

Evaluation

We evaluate the M2 paging subsystem with a VM PUP-style microbenchmark modelled on Appel and Li's 1991 ASPLOS paper. All numbers below are from a single run on the imx8x board with the system counter at 8 MHz. Each result row is emitted as a CSV line and post-processed into the plots that follow. The reservation size used in the lazy/eager and hybrid experiments is fixed at 1000 pages.

Base costs

We start with the four primitive cost numbers, shown in Figure 1. The leftmost two columns are language-level baselines, a register-to-register integer add and a write to an already mapped page. The rightmost two are the headline VM operations, mapping then unmapping a single page and faulting in a single page through the user-mode page fault handler.
A bare add costs five nanoseconds and a touch on an already mapped page twelve nanoseconds, so the cost of installing a single page table entry under our paging code, including the explicit unmap, is around four orders of magnitude above the bare arithmetic baseline. A single user-mode page fault is another factor of forty on top of that. This gap is exactly the phenomenon Appel and Li raised in 1991, and we revisit it below in the historical comparison.

Comparison to 1991 systems

Figure 2 compares our user-mode page fault round trip to the seven systems Appel and Li tabulated in 1991. The left panel reports absolute wall clock time, the right panel reports the same number divided by each system's own add cost, which is the normalization Appel and Li argued for. Our results make their argument even stronger.
In absolute time we are the fastest of the comparison set, at 220 microseconds against a 1991 range of 300 microseconds to 3.3 milliseconds. Once we normalize by add cost, the picture inverts. Our fault costs about forty-four thousand adds, against a 1991 range of two thousand to twenty-eight thousand. CPUs have gained roughly three orders of magnitude in the intervening decades but our VM round trip has only halved, so the relative cost of a fault has actually grown. The complaint

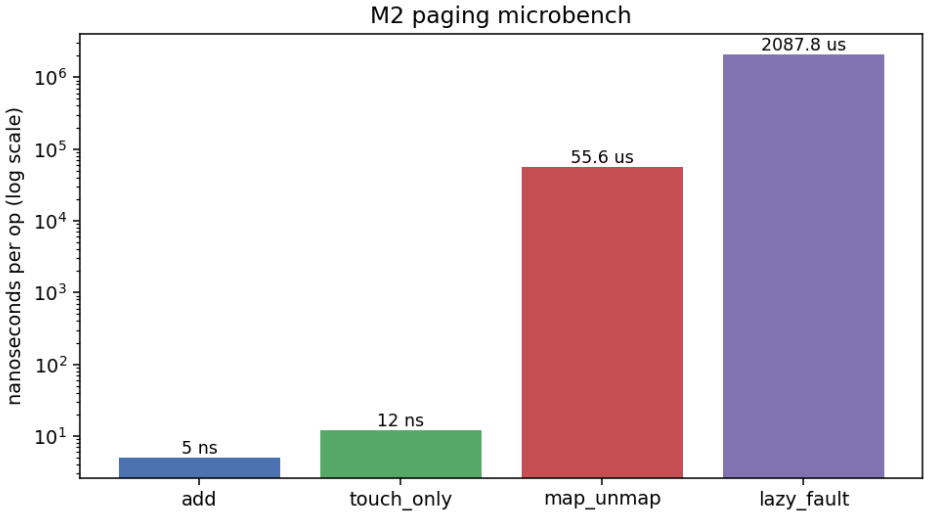


Figure 1: Per operation cost of the four primitives. Log scale on the y axis; the four bars span roughly six orders of magnitude.

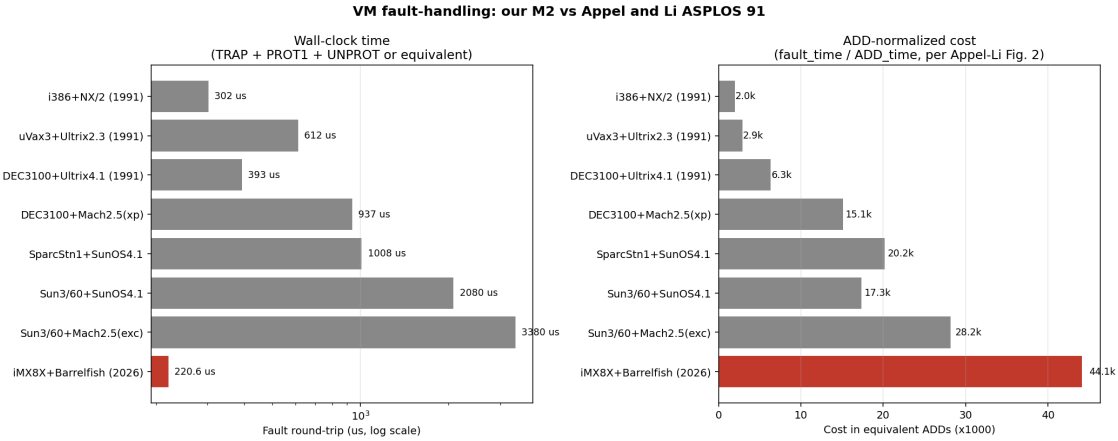


Figure 2: Page fault round trip on our system against the seven 1991 systems tabulated in Appel and Li's paper.

Appel and Li levelled in 1991 still applies, only more so. The cost of crossing into the kernel and back, even on a modern ARMv8 with a user-mode page fault handler that we wrote ourselves, has not kept up with the rest of the machine.

Allocation Strategies Across "Touch Ratios"

For the comparison between lazy and eager allocation we vary the fraction of the reserved region that is actually touched, from ten percent up to one hundred percent in steps of ten. Figure 3 shows both panels, physical frames committed on the left, total wall clock time on the right.
The frame plot reproduces the obvious story very cleanly. Lazy tracks the diagonal, committing one frame for each page that is actually touched, plus a small constant overhead from the slab allocator and the page fault path itself. Eager sits flat at one thousand frames regardless of how many of them get used, because the reservation is satisfied by a single batch map up front. At a ten percent touch ratio, eager is committing ten times more physical memory than the workload actually requires.

The time plot is the corresponding cost. Eager stays in the low tens of microseconds across the

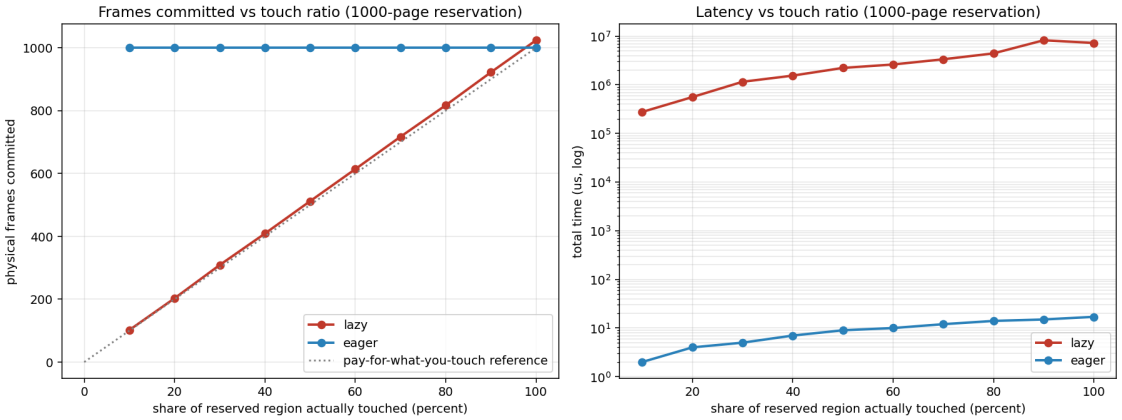


Figure 3: Lazy versus eager as the touched fraction of the reservation grows. Reservation is 1000 pages throughout.

entire sweep, dominated by the one-shot batch map. Lazy starts at about 280 milliseconds for one hundred faults and grows roughly linearly with the number of faulted pages, reaching seven seconds at one thousand pages. The two curves never cross. As long as we are willing to pay for physical memory up front, eager is two to five orders of magnitude faster than lazy on this hardware.

Hybrid Strategy Under Random Access

The natural next question is whether a strategy that pre-maps some of the reservation eagerly and lets the rest fault in lazily can beat either endpoint. Pre-mapping costs memory but eliminates faults on hits, so the answer depends on what fraction of the touches hit the pre-mapped pages.

To stay honest we drive this experiment with a random access pattern. The bench seeds the C library random number generator from the system counter and issues each touch at a uniformly random index over the whole reservation, so the access pattern carries no oracle information about which pages will turn out to be hot. A touch falls in the pre-mapped region with probability proportional to the size of that region, and hits the lazy region the rest of the time.

Figure 4 sweeps the preload region size across 0% to 100%, touching 100 random pages in each run. The two dashed reference lines are pure lazy and pure eager at the same touch count.
The curve sits within a factor of three of the pure lazy reference for every intermediate size we tested. Even with five hundred pages pre-mapped, the wall clock time only drops to about 200 milliseconds, against pure lazy’s 355 milliseconds. The cliff that finally takes the line down to nine microseconds appears only at the right edge of the sweep, where the eager region equals the full reservation and there is no lazy region to fault into. In other words, on random access the hybrid policy degenerates. Unless you pre-map essentially everything you might touch, the touches that miss the pre-mapped region still pay the full per-fault cost, and the small expected number of hits is not enough to move the total.

The narrow sweep in Figure 5 zooms into the region where the pre-mapped size and the touch count match, varying the eager region from ten to one hundred pages.
The frame panel grows monotonically from 108 to 187, well above the grey reference line that shows what the pre-mapped pages alone would cost. The gap between the curve and the reference is the lazy region’s contribution, since roughly nine out of ten random touches still miss the pre-mapped region and bring in a fresh frame. The latency panel is noisy enough that we treat the individual points with caution; the average sits around 700 milliseconds and shows no clean trend across the sweep. The committed frame count grows by about ten pages for each ten-page increase in the pre-mapped region, but the time stays in the same band as pure lazy.

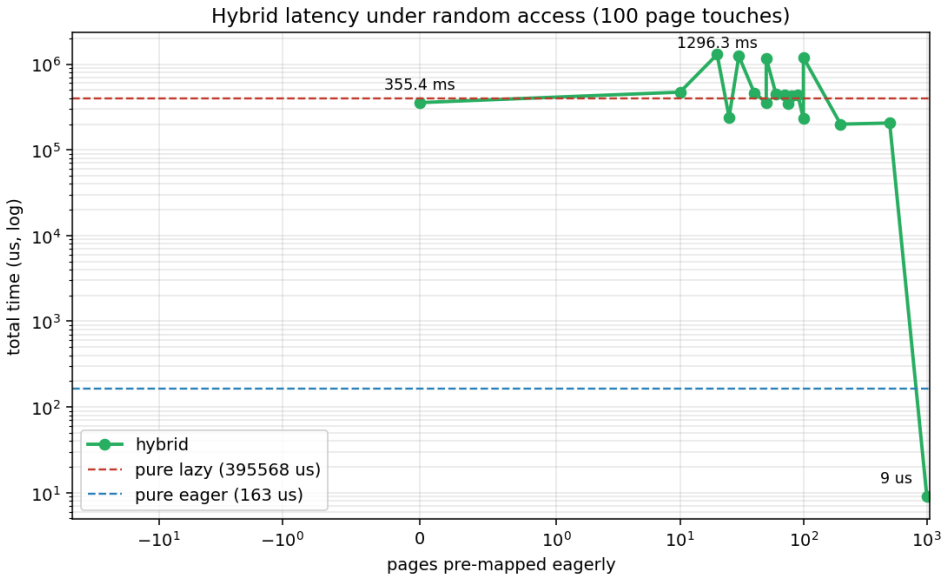


Figure 4: Hybrid latency as the eager region grows from zero to the full reservation. Random touch pattern, 100 touches.

Frames Versus Time Trade-off

Figure 6 brings the three strategies onto a single trade-off plot. The bar chart on the left shows physical frames committed at a hundred-touch workload, with the hybrid case shown for three representative pre-map sizes. The scatter on the right places each strategy in the two-dimensional space of memory cost versus time cost.
Eager sits alone in the upper left of the scatter, which means it commits the full thousand frames but the touches themselves take only two microseconds because every PTE is already installed. Lazy and the three hybrid points cluster in the lower right, all at roughly one hundred to two hundred frames and all at hundreds of milliseconds of wall clock time. Lazy is the best of that cluster, committing the fewest frames and running in the shortest time of any non-eager option. None of the hybrid points dominate it on either axis.

The practical conclusion for our system is that there are two sensible policies and no useful middle ground. If memory pressure is the binding constraint, lazy is the right choice; the per-fault cost is high but the physical commit is exactly what the workload uses. If latency is the binding constraint and the reservation can be fully committed up front, eager wins by four orders of magnitude because it amortizes the per-page cost into a single batch map. Hybrid is dominated by lazy in the random-access regime we measured. It would only become attractive if the access pattern were known in advance, in which case the pre-mapped region could be sized to cover exactly the hot pages and the experiment would collapse back into a small eager allocation.

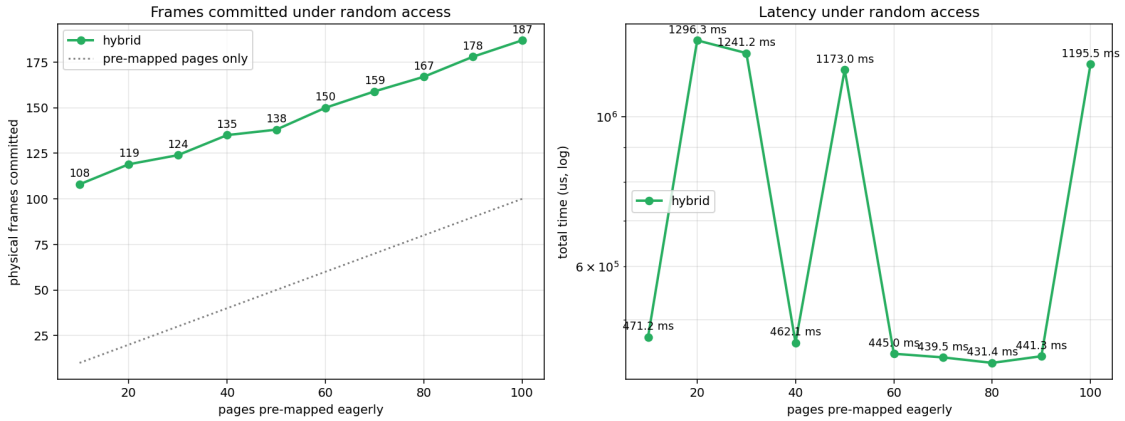


Figure 5: Hybrid frames and latency for small eager regions, with a fixed 100 touch random workload over a 1000 page reservation.

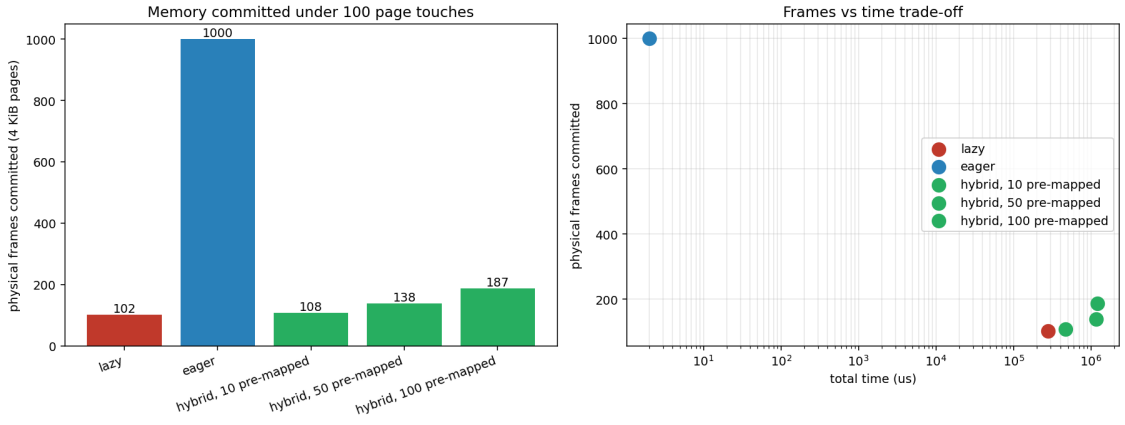


Figure 6: Eager pays ten times the memory for near zero touch latency; lazy and hybrid land in the same lower band of memory but pay hundreds of milliseconds.

M4: Core-Local Message Passing

Author: Shane Nelson

The goal of this milestone is to enable threads to access services such as process management and memory allocation via intra-core remote procedure calls.

Architecture overview

Our M4 architecture is split cleanly between our client side request API contained in `include/aos/aos_rpc.h` and our server side handlers contained in `usr/init/rpc_server.c`. This section will focus on only the content in these files relevant for LMP, there are many additional details in the source for these files, but these pertain to M6 and as such are not discussed here. Figure 7 below is a diagram illustrating the overall LMP subsystem architecture.

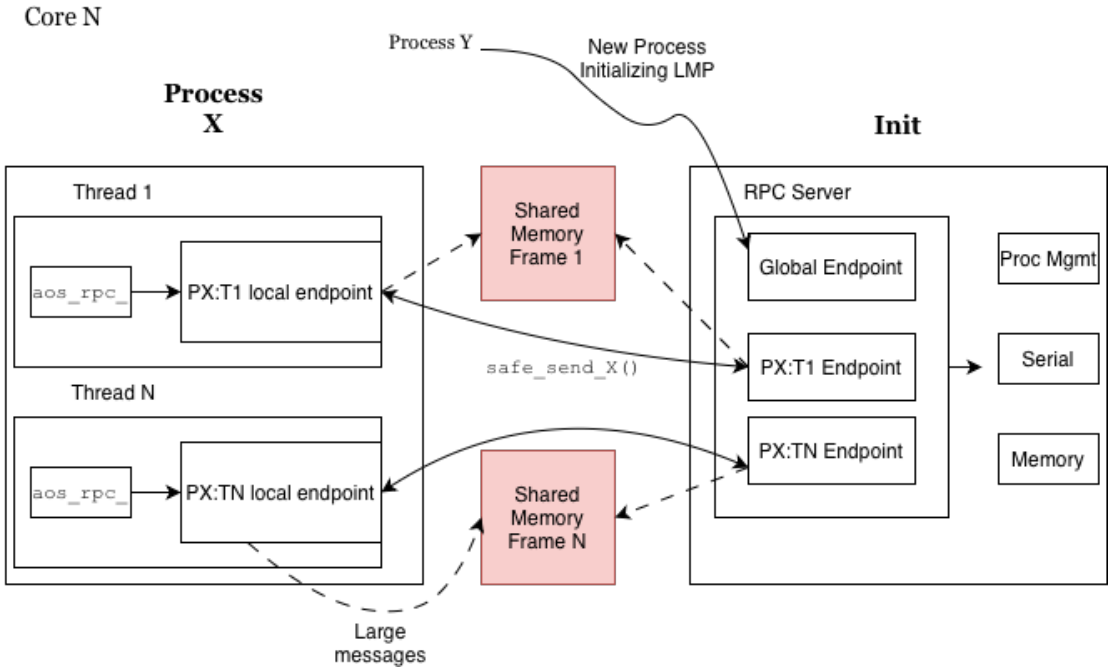


Figure 7: Overall LMP subsystem architecture.

Client-side

We choose to implement per-thread lmp channels in order to increase the available concurrency in our system. This allows any thread to wait on a process without blocking the progress of other threads, and a similar optimization was implemented by Linux (this behavior can be achieved by passing `__WNOTHREAD` to `waitid`). This also has the benefit of making synchronization trivial but has the down-side of added initialization complexity.

Thread-Local state

Each thread keeps a `struct aos_rpc` which stores metadata about the channel, such as the channel itself and the data pertaining to shared memory. Each thread also keeps a distinct waitset which allows the underlying system to correctly wakeup threads when messages have arrived on their channel. The exposed `get_X_channel()` and `get_default_waitset()` functions are wired to use this thread-local state.

Channel initialization

Each thread bootstraps its channel lazily on first use via a hook registered with the threading library. The initialization follows a two-step handshake. First the thread creates a temporary `lmp_chan` aimed at `cap_initep`, init's well-known global endpoint, creates a new local endpoint, and sends an `AOS_RPC_MSG_HANDSHAKE` message carrying its own local endpoint capability. Init responds by allocating a dedicated per-thread channel and replying with that channel's endpoint capability. The thread then replaces its `remote_cap` with this new per-thread endpoint, and all subsequent communication bypasses the global endpoint entirely.

Synchronous send and receive

All RPCs follow a strict synchronous pattern, the client sends a tagged message via `safe_sendN()` (a `lmp_chan_send()` wrapper which transparently retries on a transient error), then blocks on `recv_msg()` until a reply arrives. `recv_msg()` registers a one-shot callback on the thread's private waitset and spins on `event_dispatch()` until the callback fires and marks the slot as done. Because each thread has its own waitset and dedicated channel, threads never race to receive each other's replies.

Server-side

In order to facilitate per-thread channels our server maintains multiple LMP endpoints. One endpoint is a dedicated global listening endpoint which handles the initialization of new channels. The server also maintains one endpoint per user process thread, created after handshaking.

Handshake protocol

Client processes are initialized with a capability `cap_selfep` when they are initially dispatched. This capability points to the global endpoint. The client mints a new endpoint and sends it to init over the listening channel. This channel is set up to only enter in `init_handshake_handler`. Once in the handler the server allocates a new local endpoint to communicate with the child using the endpoint it receives, creates an lmp channel out of it and then sends this newly created endpoint back to the client in a `AOS_RPC_MSG_ACK` tagged message. The server then creates a `struct client_state` which stores the per-client metadata such as the `lmp_chan` and information pertaining to the shared memory frame used for large messages. Finally the server registers another function `init_recv_handler` (the general purpose entry) as the entry point for the newly created endpoint, and provides a closure with the created `struct client_state`. Now both sides can communicate on the dedicated channel.

General purpose serving

When a message is received on a dedicated channel, the system calls `init_recv_handler` and provides the closure created in the handshake. This function reads the message type from the first byte of the message and calls the correct handler for the desired request type. The handler which is called is determined using a table mapping message types (defined in `aos_rpc.h`) to function pointers. Once in the handler, the server will execute the desired command locally, and send the client back either an acknowledgement message or the result of the RPC.

Special case: `aos_rpc_proc_wait()`

When a thread wants to wait on a process, it must be notified via RPC when the process exits. To facilitate this, the process management function `prod_mgmt_register_wait()` maintains a linked list of `struct waiter_node` for each process which holds metadata about the client channel. When a client calls the wait rpc function, its channel is added to this list and it is not sent a response. This blocks the thread because it is waiting in a synchronous receive loop which blocks it until a message is received on the channel. When another thread (or `libc_exit`) terminates the waitee process, `proc_mgmt_terminated` is called. This function walks the process's `waiter_node` linked list and sends an rpc message to the channel stored therein. This brings the thread out of its receive loop and unblocks it.

Shared memory for large messages

In order to reduce the latency for large messages, we implement a shared memory frame to pass such messages. Without shared memory we would be forced to break large messages into cacheline-sized messages and send them one by one. This likely has large latency implications (although we did not explore testing this). To implement this lazily, before a client sends a request which requires more than one cacheline it undergoes a shared memory handshake process with init as follows:

1. Client tries to request an operation that requires a request or response larger than one cacheline and sees that `struct aos_rpc.shm_vaddr` is NULL.
2. Client allocates a physical memory frame.
3. Client maps this frame into its virtual memory space and updates the `struct aos_rpc.shm_vaddr` and `struct aos_rpc.shm_size` fields.
4. Client sends frame capability to init in a `SHMEM_SETUP` message.
5. Init maps the capability into its virtual address space and updates the corresponding `struct client_state` fields for the client.
6. Init sends back a `SHMEM_ACK` message.
7. Client proceeds to whichever operation was in progress.

Optimizations and performance evaluation

Performance Goals and Optimizations

Our performance goals for our optimizations of milestone 4 were (1) reduce the latency of multi-threaded RPC, and (2) reduce the memory usage of our shared-memory approach.

1. Latency Reduction for Multithreaded LMP

We anticipate multithreaded workloads that make heavy usage of the local RPC infrastructure to perform very poorly using a naive mutual-exclusion locking based approach to synchronization. For this reason we introduced per-thread LMP channels which allows for greater concurrency in the face of load. The implementation of this design is already discussed at length above, see Client-Side for details. We test the efficacy of this optimization below, see Performance Optimization.

2. Memory Usage Reduction of Large Message Infrastructure

One of the main drawbacks of the prior optimization is the memory usage due to each channel having a frame allocated for shared memory communication. This memory usage obviously increases when the number of channels increases, as it does in our prior optimization from one per process to one per thread. In a commercial system running many thousands of user-level threads, this presents a serious problem. To get around this we lazily allocate shared memory frames only when

an operation that needs a large message cannot fit the message in a certain (tunable) number of cachelines. This means that only a small subset of threads (those using RPC to pass large strings or those calling "ps" frequently) ever get a shared physical memory frame allocated. Some further optimizations would be to intelligently deallocate frames when not in use, perhaps using a variant of our `aos_rpc` API which tells the system to reclaim the frame after receipt of the data, and to make a variant of the API which allows the system to run in "memory-constrained" mode which will opt to send cachelines message by message rather than using any shared memory. We do not implement these optimizations, and importantly, due to time have not implemented memory-usage benchmarking to judge the performance of the optimization we did implement.

Performance evaluation

Microbenchmarks

To facilitate comparing a non-optimized LMP implementation to our optimized implementation, we created three microbenchmarks:

1. **RTT: Round Trip Time.**

   This benchmark is designed to test the round trip cycle latency for a simple `aos_rpc_send_number()` operation. It is single threaded and is initially run 20 times to warm up microarchitectural structures, then 200 times each collecting a cycle latency measurement.

2. **Stress: Shared Memory RPC Stress Test.**

   This benchmark measures RPC latency under high multithreaded contention. 16 worker threads are spawned, they each run a warm-up phase, then all at once they each try to execute 50 `aos_rpc_send_string` operations, recording the latency of each operation. Thus we measure how LMP and our shared memory approach scale under high load.

3. **MTW: MultiThreaded Wait.**

   This benchmark evaluates the effect of a long-lived blocking RPC on the throughput of concurrent RPC activity. Four worker threads simultaneously send various `aos_rpc` requests to init. When run in waiting mode, another thread first waits on a separate process, then the worker threads begin. When run in baseline mode, no such waiter exists. We expect our per-thread LMP implementation to see almost no drop off between modes, while the latency of the naive implementation should be bounded by the latency of the waitee process.

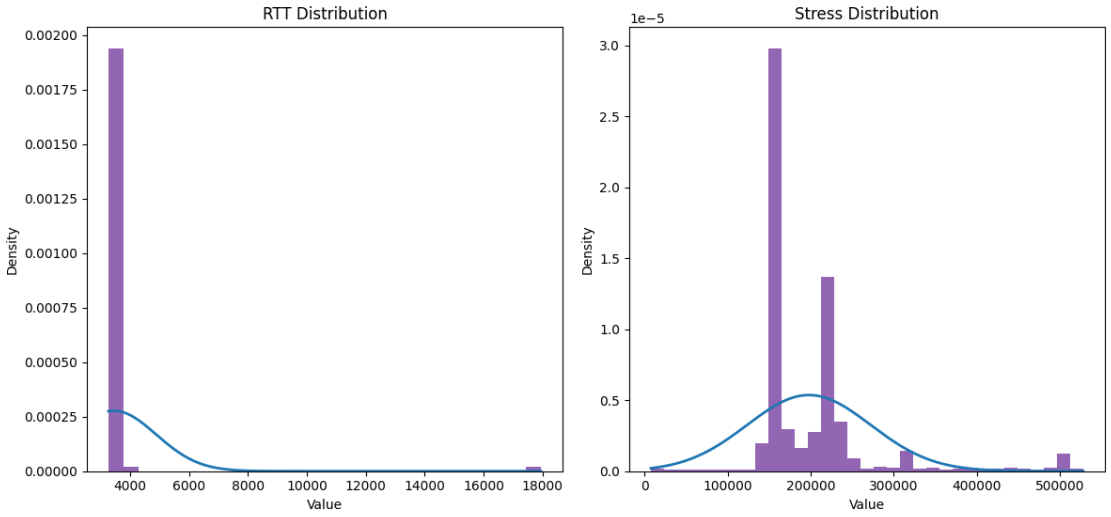
Results

We see a modest improvement in median Stress, insignificant difference in RTT, and as expected massive wins in MTW. Figure 8 displays the Stress and RTT results for the un-optimized build and Figure 9 displays the same results for our optimized build. RTT shows a significant increase in mean cycles, however this is swayed by a few outliers that are observed consistently at the end of the benchmark. Median (P50) RTT results show a slight increase for our optimized build. I believe this to be due to a slight reduction in cacheability of perthread `struct aos_rpc` compared to per-process (although this is merely a hypothesis). Interestingly, the RTT distribution for the optimized group is consistently bimodal. We observe a periodic doubling in latency for single-threaded operations, and at present we cannot explain this phenomenon, this is an important area for future investigation.

Our multithreaded stress test (Figures 8 & 9 right panel) shows that our optimized build required at mean only 78% as many cycles as the unoptimized build, and at median only 89% of cycles. Of importance is also the behavior of the tails where the unoptimized build had a significant number of latencies 2x higher than the maximum optimized latency. This explains the mean-median discrepancy. We find this to be the strongest result for our implementation as multithreaded IPC-heavy workloads are readily imaginable.

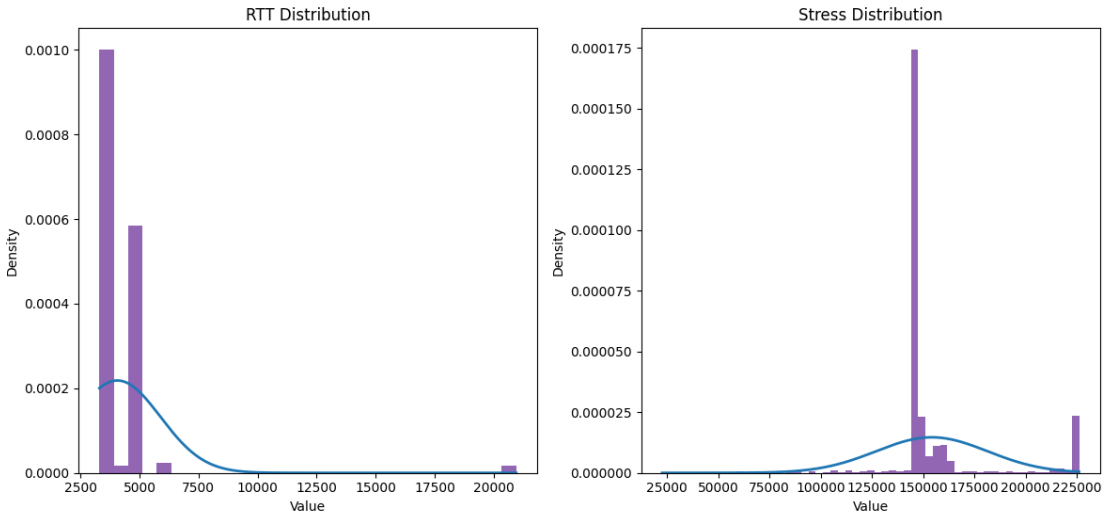
Finally, the results for our multithreaded waiter benchmark (Figure 10) reflect our initial expectations exactly. Our optimized build saw only a 26% increase in latency with a blocked waiter,

while the naive build saw a $> 1000\%$ gain in latency, bounded by the time-to-exit of the waitee process. This result reflects our initial motivation for building this optimization. It may seem heavily cherry-picked, but you could imagine an application such as a shell having such behavior.



	Mean	P50 (Median)	P90
RTT	3448.64	3286.0	3318.8
Stress	197798.64	164679.0	243952.7

Figure 8: Naive implementation benchmark results for a single-threaded number passing benchmark, labeled RTT, and a multithreaded string passing benchmark, labeled stress.



	Mean	P50 (Median)	P90
RTT	4058.51	3323.0	4759.6
Stress	154180.27	146525.0	203783.8

Figure 9: Per-thread implementation benchmark results for a single-threaded number passing benchmark, labeled RTT, and a multithreaded string passing benchmark, labeled stress. As can be seen the overhead to create a perthread channel (evident in the median RTT statistic) is negligible compared to the baseline. The per-thread implementation sees significant gains in median multithreaded latency.

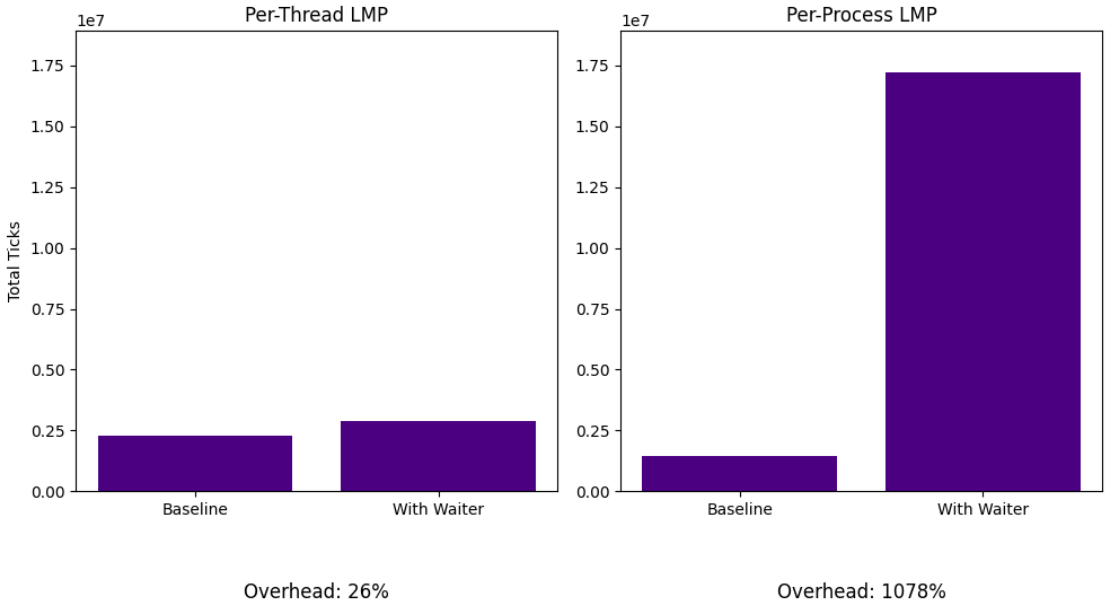


Figure 10: Performance drop-off from running a multithreaded workload (MTW) without a thread waiting on a process to finish, to running one with a thread waiting. Per process lmp channels perform $>$ 1000% worse while a thread is waiting, whereas per-thread channels see only a modest 26% cycle gain.

Development process

What I will call phase 1 began with Sahil writing the client side state and handlers in `aos_rpc.c`/`aos_rpc.h` as well as the original initialization code in `barrelfish_initonthread()`. Allan then wrote the serverside entry point as well as wrappers for all of the per-request handlers and the implementations of the memory and serial services. Shane then finished the naive implementation by finishing the initialization, server, and client functionality, and implementing naive mutex-based synchronization and shared memory for large messages. At this point we were passing all of the `m4_grading` tests.

Phase 2 began after a conversation on Ed with Simon which motivated Shane to implement the optimizations above. This involved reworking and hardening the implementation in phase 1, as well as changing the protocol to handle per-thread LMP channels. This required touching every LMP-related file as well as the threading library and was difficult to get running, requiring debugging with Simon in office hours. Using AI tooling tests were written for the optimization, which it passed.

The final phase began when collecting data for this report, where Shane (with the help of AI tooling) reverted to an unoptimized version of the code. Shane then implemented 3 benchmarks, collected results, and wrote the M4 section of the report.

M5: Multicore

Section Author: Sahil Gupta

Architecture Overview

For multicore, our architecture is quite simple and straightforward. Cores are brought up by invoking the correct PSCI functions through the kernel (the same method is used for shutdown, reset, and reboot, as discussed later). Communication between cores for this specific milestone is handled through the `initial_urpc` struct, which lives in main memory. The relevant code for this milestone can be found in the following files:

- `usr/init/coreboot.c`
- `usr/init/coreboot.h`
- `usr/init/main.c`
- `usr/init/mem_alloc.c`
- `usr/init/mem_alloc.h`

Bonus work is discussed in the Suspend, Resume, Shutdown, and Reboot section. Below is a simple diagram outlining the process of booting the different cores and communicating with them.

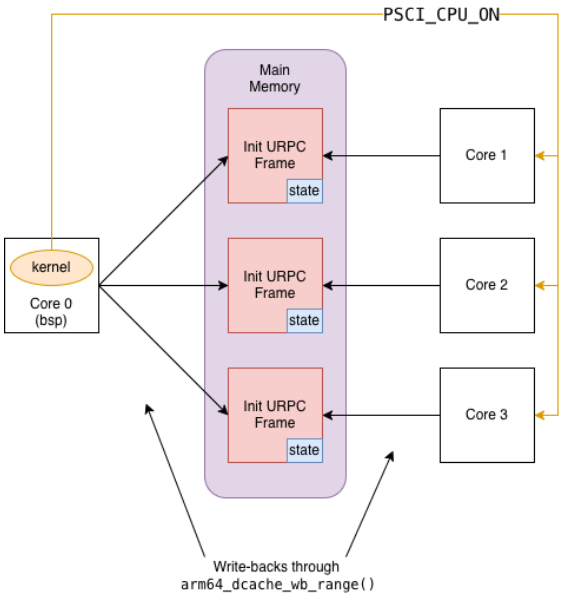


Figure 11: Multicore Architecture

Initial URPC Frame

Before discussing any of the code written for this milestone, or any of the functionality, we need to discuss the core struct for communicating with the booted cores, which contains some new fields.

```
struct initial_urpc {
    genpaddr_t            bootinfo_frame_base;
    gensize_t             bootinfo_frame_bytes;
    genpaddr_t            mmstr_frame_base;
    gensize_t             mmstr_frame_bytes;
    genpaddr_t            ram_base;
    gensize_t             ram_bytes;
    volatile corestate_t  state;
    uint32_t              active_cores_mask;
    struct {
        genpaddr_t base;
        gensize_t  bytes;
    } peer_rings[COREBOOT_MAX_CORES];
};
```

The fields `active_cores_mask` and `peer_rings` are used for User-Level Message Passing and can be ignored for now. The rest of the fields are used to boot up different cores, pass memory from the BSP core, and communicate the state of the booted core.

`bootinfo_frame_base`, `bootinfo_frame_bytes` Used to pass the booted core information about the `bootinfo` struct so it can be forged on the booted core. The `bootinfo` struct contains memory layout information crucial for the core to allocate memory.

`mmstr_frame_base`, `mmstr_frame_bytes` Passed to the booted core to forge the relevant capability, used to discover module names for processes spawned on that core.

`ram_base`, `ram_bytes` A region of RAM donated from the BSP core to give the newly spawned core memory to work with.

`state` A synchronization flag with several states, modifiable by either the BSP core or the spawned core to communicate boot progress.

Booting Another Core

Now we can talk about the process of spawning a new core. The following steps are executed by the BSP core at startup, which invokes `coreboot_boot_core` on every core (we support booting every core on the Toradex board).

1. Allocate and fill in the Initial URPC frame, then flush it to main memory so the booted core can see it
2. Locate the correct binaries for the CPU driver, boot driver, and the `init` program
3. Load the relevant ELFs and relocate them
4. Allocate a new KCB and fill in the relevant fields
5. Call `invoke_monitor_spawn_core`, which in turn invokes `PSCI_CPU_ON` through the kernel to spawn the core
6. Wait for the spawned core to signal it is ready

On the spawned core, execution begins in the `app_main` function in `usr/init/main.c`, which forges the correct capabilities and signals readiness to the BSP core by setting the `state` field.

Multicore Memory Management

Due to the design of Barrelfish, managing memory across multiple cores is non-trivial, as each core must know what memory it is responsible for. In our design, the BSP core carves out a fixed region of memory to hand over to the spawned core, using the `ram_*` fields of the `initial_urpc` struct and `ram_alloc`. We fix the size of this region at 128 MiB which is large enough for the new core to boot, but small enough not to starve the BSP core. We arrived at this value through empirical testing, and note that other values may be more appropriate depending on the multicore workload.

Suspend, Resume, Shutdown, and Reboot

In this milestone, we support the ability to suspend, resume, shutdown, and reboot a core (though reboot is not fully functional), implemented primarily through the `initial_urpc` struct. First, we need to revisit the `state` field. Its type is shown below:

```
typedef enum corestate {
    CORE_STATE_UNKNOWN,
    CORE_STATE_OFF,
    CORE_STATE_RUNNING,
    CORE_STATE_SLEEPING,
} corestate_t;
```

When the BSP core spawns a new core, the field is initialized to `CORE_STATE_UNKNOWN` and the newly spawned core then sets it to `CORE_STATE_RUNNING` once it is ready. This pattern lets a remote core detect when its state needs to change based on a signal from another core.

Suspend and shutdown are the simplest functions. Since equivalent PSCI functions exist, we create monitor entry points into the kernel and invoke them directly. When the BSP core instructs a remote core to suspend or shut down, it updates the `state` field in the URPC struct (`CORE_STATE_SLEEPING` for suspend, `CORE_STATE_OFF` for shutdown). The remote core polls this field in `app_main` and acts accordingly. This same signaling pattern is used for the other lifecycle functions.

For resume, there is no direct PSCI function. According to the PSCI specification, a suspended core requires a wakeup event to resume. We use a Software Generated Interrupt (SGI) as the wakeup event, implemented via a monitor syscall that invokes `gic_raise_softirq` on the suspended core from the kernel.

For reboot, we do not support a core rebooting itself and the only supported path is the BSP core rebooting a remote core. We accomplish this by first signaling the remote core to shut down, then invoking `coreboot_boot_core` on the same core ID.

Development Process

This milestone began with implementing the main functionality in `coreboot_boot_core` in `usr/init/coreboot.c`, where we allocated all the necessary data structures and set up the shared URPC frame. In this part, we also enabled booting all cores on the Toradex board by modifying `usr/init/main.c`.

We then implemented memory allocation on remote cores, which involved allocating a large chunk of memory through `ram_alloc` and passing it through the URPC frame. We also needed a way to initialize the allocator from a forged capability, which required us to implement `initialize_ram_alloc_from_cap` in `usr/init/mem_alloc.c`.

Finally, we implemented all the core lifecycle functions and wrote sample test cases to demonstrate the functionality.

M6: User-Level Message Passing

Author: Shane Nelson

The goal of this milestone is to enable inter-core communication between processes.

Architecture overview

The UMP subsystem connects the init processes of each core via a shared-memory frame treated as two circular ring buffers. Processes always communicate with their local init using LMP. When a process issues an `aos_rpc` call that targets a service on a remote core, local init intercepts the LMP message and forwards it over UMP to the appropriate remote init, hence all messages require an init interception. This is intended as a simplification, but also is logical because all services in the API are served by init. We extend this original API to enable arbitrary process-to-process piping as required by a shell. The Archicture Overview section only discusses the infrastructure needed to implement the original API, see Extension for Shell for discussion on the extension. Figure 12 below illustrates the base UMP subsystem.

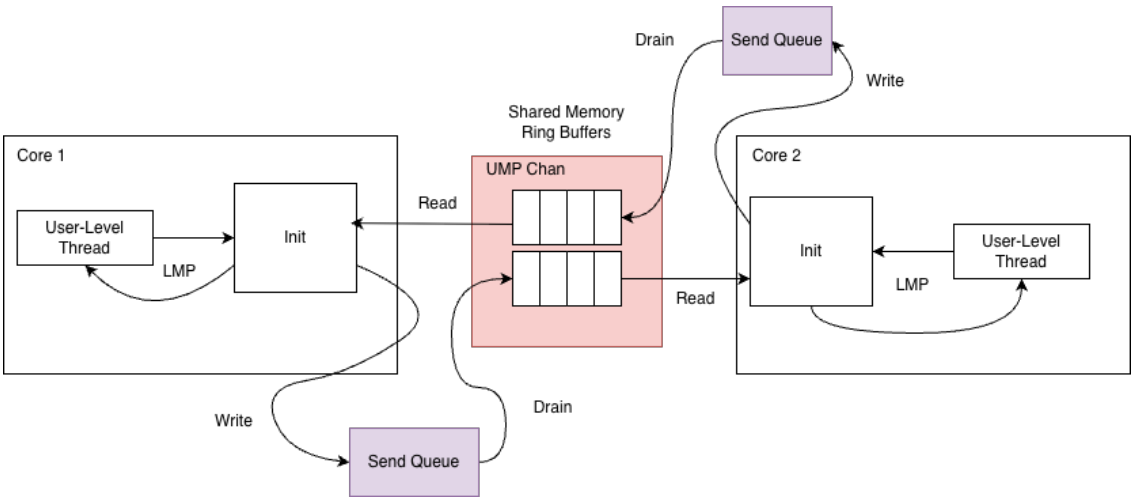


Figure 12: Abstracted view of the RPC system.

Initializing UMP channels

Before booting any APP cores, BSP allocates a frame for each pairwise core-to-core UMP channel and stores the physical address of each frame. When booting an APP core, BSP inserts the address of each of the APP core's UMP frames (one for every peer core) in the `initial_urpc` struct along with information about which cores are online. Inside the APP core's initialization process, it maps each of these frames into its virtual address space and initializes the channels for each core that is currently running. BSP then sends a `AOS_RPC_MSG_CORE_UP` over UMP to each APP core that was previously running, telling it that a new core is online. These APP cores initialize their end of the UMP channel with the new core. As part of the initialization process, each side's init

sets a deffered timer, which periodically allows it to check for received UMP messages. At this point both sides have initialized the channel and it is ready for use.

Memory layout of UMP channel

A UMP channel is a shared-memory frame divided into two circular ring buffers. Each side of the connection writes into one buffer and reads from the other. Each buffer is a sequence of cache-line-sized *slots* with domains communicating by writing and reading slots. We rely on the MOESI cache coherence policy to synchronize writes and reads to these slots.

Because the ring buffer slots are a finite shared resource, a send may fail if the buffer is full requiring the sender to block. To decouple *logical* sends from *physical* availability of ring slots, each channel maintains a dynamically-allocated send queue (linked list) of messages waiting to be sent. When a physical send would block, the message is instead appended to this queue. A periodic timer drains the queue by calling `drain_send_queue`, which flushes enqueued entries into the ring as slots become free. From the caller's perspective, a send always succeeds immediately. The actual placement into shared memory is deferred transparently, allowing the sender to proceed handling more LMP requests.

For most RPC operations, all data fits in a single slot. However, some requests and replies exceed one cache line and must be split across multiple slots. These *multi-slot* messages consist of a *header slot* followed by one or more *continuation slots*. The header encodes the message tag, the total number of words in the message, and the first five data words. Continuation slots carry the remaining data words. In one byte the message tag encodes the RPC operation and the message's role in the protocol: request header, reply header, request continuation, or reply continuation

To reassemble multi-slot messages before dispatching them, we introduce two auxiliary tables. `pending_recv` is used by the receiver to buffer and assemble incoming multi-slot requests until all continuation slots have arrived. `pending_replies` is the symmetric table used by the requestor to buffer incoming multi-slot replies. These tables are statically sized and allocated. Importantly, the message tag also encodes the index into the tables (which are symmetrically indexed) as a means of uniquely identifying to which transaction a particular message belongs. This ensures that messages are assembled and dispatched atomically even in the presence of non-atomic sends (messages for different operations may interleave). When a message is fully constructed, it is "dispatched", the actual behavior of the dispatch depends on if the domain is the original requestor (i.e. write end) or the receiver (i.e. read end) for the message.

Handling incoming messages

TODO: talk about deffered timer and how it woeks a bit, then jump into this.
When the timer set on init fires, execution is started in the timer handler function. The timer handler function, `ump_poll_handler()`, loops through all UMP channels and processes any messages which may have been received. Based on the message type, the handler does the following:

Request Header:

1. If the entire message fits in the header, immediately dispatch into the correct header based on the request type.
2. If the entire message does not fit in the header, allocate a slot in the `pending_recv` table storing the size of the message, how many bytes weve received so far and the type of the request.

Request Continuation:

1. If the message is now complete, free the `pending_recv` entry and dispatch.
2. Otherwise update how many bytes we still need to receive.

Reply Header:

1. If the entire message fits in the header, immediately respond to the LMP client. Deallocate the previously allocated `pending_replies` entry. (See Requestor for discussion on how this was allocated).
2. If the entire message does not fit in the header, update the `pending_replies` entry

Reply Continuation:

1. If the reply is now complete, respond to the LMP client. Deallocate the previously allocated `pending_replies` entry.
2. Otherwise update how many bytes we still need to receive.

Requestor

The requestor in the UMP protocol is the init domain which initiates an RPC call on behalf of a process on its core. An init may act as both Requestor and Receiver for different messages concurrently, but each message has one requestor and receiver.

When init receives an LMP request, it determines the target core. For `aos_rpc_proc_spawn_with_cmdline()`, the target core is specified as an argument. For other process management functions, the process ID of the target process is supplied as an argument. We encode the target core in the PID. If a request targets a remote core init does as follows:

1. Ensure that the target core is running, if not return an error.
2. Allocate an entry in `pending_replies` to buffer any replies the remote core may send.
3. If there are no entries available in the buffer, send a transient error to the client. The client will then prompt the kernel to execute a context switch. When the client is rescheduled it retries the operation. All of this occurs transparently so the programmer must not explicitly handle such errors.
4. Parse LMP arguments into required header and cont messages.
5. Enqueue messages into the send queue and attempt to drain the queue, sending the messages on the UMP channel to the remote core.
6. Handle other business while the LMP client maintains blocked waiting for a reply.

When the timer fires, `ump_poll_handler` checks for replies. If the reply is ready the requestor translates the UMP reply into an LMP reply. The client LMP channel is stored in the `pending_replies` entry, thus init knows where to send the LMP reply.

Facilitating global operations

There are certain operations such as kill all and ps which require init to communicate with the init on every core rather than only one core's init. For such operations we find a `pending_replies` entry for each core that is online, making note that the entry is for a global operation. We then send UMP requests to each UMP channel in the exact same method as above. In order to track the incoming replies we introduce another static datastructure `ump_fanout`. This struct holds information used to compile the final result as well as information about which cores still need to respond. Once all responses have been gathered we compile the results and reply to the client via the lmp channel stored in `ump_fanout`. There is only one `struct ump_fanout` for each init image so each core can only have one outstanding global operation at any point.

Receiver

The receiver is much simpler than the requestor. Once the entire request is compiled via `ump_poll_handler` the correct handler is called for the required rpc operation. We pass the handler a newly created `client_state` struct which encodes that this is a UMP request. Once in the handler we execute the rpc command locally. We then assemble the UMP reply into a header and any necessary continuation messages and attempt to send them over the UMP channel.

Concurrent UMP operations

The `pending_replies` buffer mentioned above allows our system to handle multiple concurrent UMP transactions. A naive implementation may elect not to have such a buffer and instead only allow one outstanding UMP transaction at any point in time. This has the benefit of being simple to implement and also implicitly ensuring that send and receive pairs are atomic (a sender can be ensured that the response it receives will be its response). To test the performance benefits of allowing concurrent UMP transactions, we wrote a small microbenchmark. This benchmark spawns 16 threads, each performing one cross-core rpc operation, and records the total time for all threads to complete (this runs 50 times to get a large enough sample). We simulate the naive approach with a one-entry `pending_replies`. Figure 13 below shows the results of this benchmark.

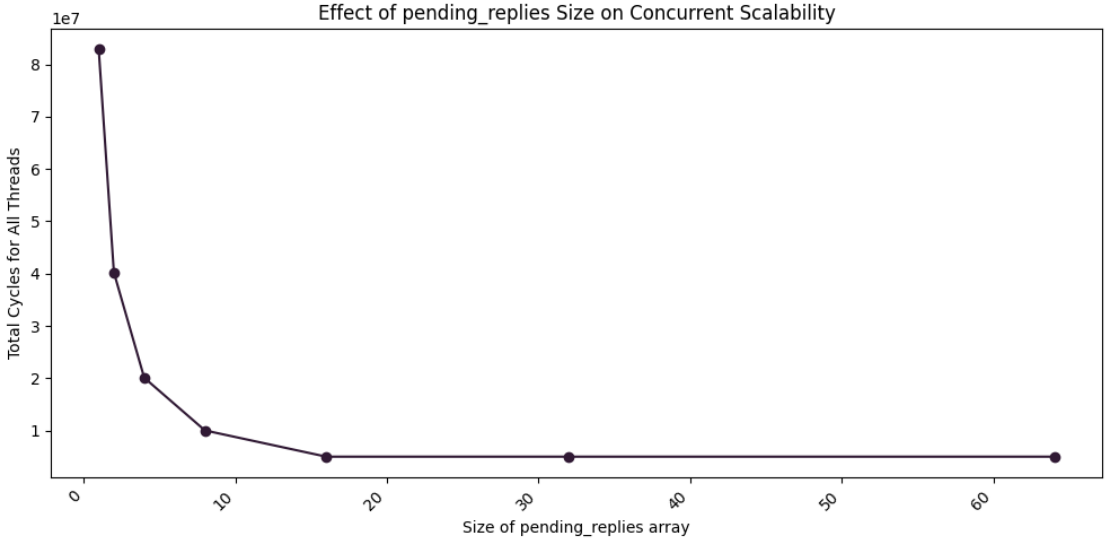


Figure 13: Results show linear speedup in the number of slots in `pending_replies`, until there are enough slots for each thread at which point there are no additional returns.

The results are exactly as expected, performance scales linearly for each additional `pending_replies` slot added. Once there is a slot for every thread the benchmark spawns [corollary: outstanding UMP request] there are no more performance gains.

This leads to an important performance invariant, in order to have optimal concurrent performance, the UMP service must be able to have a `pending_replies` and `pending_recv` slot for every concurrent UMP operation. Because LMP is synchronous, this means that there need only be a `pending_replies` slot for each thread for maximum performace under high contention. We do not implement this because depending on implementation it would impose necessary communication between init domains on how many threads there are or best case requires increased complexity in tracking operations. We hypothesize this maximal approach is not necessary because most workloads do not have every thread simultaneously communicating cross-core. We we choose an approach (64 slots) which allows for enough concurrency without complicating the protocol and design. Future work may elect to implement a dynamic allocation scheme or otherwise allow the threading library (and perhaps user) to scale the available concurrency in the system.

Development process

Allan originally implemented the infrastructure to set up a `struct ump_chan` as a set of two ring buffers and write and read from them synchronously. Shane then planned the UMP extension architecture, detailing the necessary changes to `rpc_server.c` as well as `ump_chan.c` and related headers. Shane and Sahil then implemented and debugged all the necessary changes. Shane then wrote this section.

M7: Shell

The goal of this milestone is to enable a user to interact with the system.

Command processing

Background tasks

TODO: without the & call join on the spawned process(es), else just let the process go

for background tasks we might want to replicate bash behavior where it sends the process output with like [n]: ... if you know what I mean. If we do this then we need to redirect the output of the background task to a file that we have prepended with this, then cat the file. We should have a call about this

RamFS file system support

We route all RamFS file operations through the `init` process. Each core has its own in-memory file system. In order to support this we add file system functionality to our RPC implementation. We add the following calls to the `aos_rpc` API:

```
errval_t aos_rpc_fs_open(struct aos_rpc *chan, const char *path, int
    flags,
                         fs_fhandle_t *ret_handle);

errval_t aos_rpc_fs_close(struct aos_rpc *chan, fs_fhandle_t handle);

errval_t aos_rpc_fs_read(struct aos_rpc *chan, fs_fhandle_t handle,
                         void *buf, size_t nbytes, size_t *bytes_read);

errval_t aos_rpc_fs_write(struct aos_rpc *chan, fs_fhandle_t handle,
                          const void *buf, size_t nbytes, size_t *
                              bytes_written);

errval_t aos_rpc_fs_seek(struct aos_rpc *chan, fs_fhandle_t handle,
                         enum fs_seekpos whence, off_t offset);

errval_t aos_rpc_fs_stat(struct aos_rpc *chan, fs_fhandle_t handle,
                         struct fs_fileinfo *info);

errval_t aos_rpc_fs_mkdir(struct aos_rpc *chan, const char *path);

errval_t aos_rpc_fs_rmdir(struct aos_rpc *chan, const char *path);

errval_t aos_rpc_fs_remove(struct aos_rpc *chan, const char *path);

errval_t aos_rpc_fs_opendir(struct aos_rpc *chan, const char *path,
                            fs_fhandle_t *ret_handle);
```

```
errval_t aos_rpc_fs_closedir(struct aos_rpc *chan, fs_fhandle_t handle);

errval_t aos_rpc_fs_readdir(struct aos_rpc *chan, fs_fhandle_t handle,
                            char **ret_name, struct fs_fileinfo *info);
```

The corresponding handlers are added to `rpc_server.c` and message tags to `aos_rpc.h`. The shell (or user process) can specify specify the flags which they want the file to be opened with, and can embedd the core which they prefer to create the file on by passing `FS_OPEN_ON_CORE(core) | FS_OPEN_CREATE` to `aos_rpc_fs_open()` as the `flags` argument. This is intended to mirror the spawn procedure of allowing a user to specify the core to spawn the procedure on. This works in the same way a shell spawn works (`$ run process --core n`), via the `--core` command line argument like so, `$ touch file-name --core n`. Any process can access files on any core, this is implemented with UMP redirection. To facilitate demultiplexing file operations to the correct core, we embed the core in the `fs_handle_t` in the same manner that the core of a process is embedded in its pid. Via this mechanism, `init` can route the operation to the correct core without intervention from the user. One important constraint is that a user must supply the `FS_OPEN_ON_CORE(core)` flag to access cross core files (even when not creating them). This is because file systems on different cores do not share a namespace. This is an area for future work as the current implementation pushes some burden to the user.

Pipes and I/O redirection

To enable I/O redirection we change the spawn and RPC infrastructure. Processes are spawned with information about where their input and output flows, then `aos_rpc_serial_getchar/putchar` correctly routes characters (TODO: line granularity not characters) to the correct endpoint. Piping I/O between processes works in the following manner:

1. The shell first creates and allocates a capability for the frame:

   ```
   struct capref pipe_frame;
   size_t pipe_size;
   frame_alloc(&pipe_frame, 2 * BASE_PAGE_SIZE, &pipe_size);
   ```

2. The shell then creates an `io_config_frame` for each side of the pipe and spawns the processes:

   ```
   // This omits the reader setup but it is nearly identical (the
       reader just sets pipe_in true and pipe_out_is_dsp 0)
   struct io_config_frame cfg = {
     .pipe_out        = true,
     .pipe_out_is_bsp = 1,
     .pipe_out_bytes  = pipe_size,
   };
   aos_rpc_proc_spawn_with_io(rpc, "writer_bin", core, pipe_frame, &
       cfg, &pid1);
   ```

3. Based on `core`, the correct `init` receives this `aos_rpc` call, and parses the configuration.

4. `init` then creates a new frame for the IO config info and maps it into its virtual address space.

5. `init` then assembles a capability array with capabilities to the pipe frame to pass to the spawned processes.

6. `init` then spawns the process with `spawn_cmdline_with_caps()` which places the capabilities in the childs CSpace.

7. In the new process, `barrelfish_init_onthread()` is called which does the necessary capability operations to map the pipe frame into its VSpace.

8. This fills in a global variable identifying if I/O is piped or directed to a file.
9. `aos_rpc_serial_getchar/putchar` correctly writes to the pipe rather than calling `sys_putchar/getchar`.

For file redirection the process is much the same except, fields are simply set in the `io_config_frame` that hold `fs_handle_t` for the input and/or output file, no capability operations have to occur and are skipped with `NULL_CAP` checks.

Synchronizing serial input & output

TODO: Buffer calls to aos_rpc_serial_getchar/putchar by only committing full lines (stop when newline char is seen) in aos_rpc.c and route all calls to bsp init which can set a global flag to serialize access. Make sure only bsp init can call sys_putchar. If this doesn't work find another way (maybe in the book?)

Development process

Shell development started when Shane implemented the infrastructure needed for file system operations over RPC, as well as the code which enables spawning processes with redirected I/O.

Appendix A: Shell Commands